

SPI Driver Analysis

Prepared By: Anagha

Introduction

SPI is a high-speed synchronous communication protocol used to communicate with sensors, displays, memory devices, and communication modules. SPI is a high-speed synchronous communication protocol used to communicate with sensors, displays, memory devices, and communication modules.

Driver Architecture

The SPI driver acts as an abstraction layer between applications and SPI hardware. Configuration is performed through handle structures and dedicated APIs. The SPI driver acts as an abstraction layer between applications and SPI hardware. Configuration is performed through handle structures and dedicated APIs.

Data Structures

spi_config_t stores device mode, clock settings, frame format, clock polarity, clock phase, and software slave management options. spi_handle_t stores the peripheral address. spi_config_t stores device mode, clock settings, frame format, clock polarity, clock phase, and software slave management options. spi_handle_t stores the peripheral address.

Component	Purpose
Configuration Structure	Stores settings
Handle Structure	Stores peripheral and config
APIs	Interface to application

Register Analysis

CR1 controls SPI operation, CR2 provides additional features, SR contains status flags, and DR handles data transmission and reception. CR1 controls SPI operation, CR2 provides additional features, SR contains status flags, and DR handles data transmission and reception.

API Analysis

spi_init() configures the peripheral. spi_send_data() transmits data. spi_receive_data() receives data. spi_peripheral_control() enables or disables SPI operation. spi_init() configures the peripheral. spi_send_data() transmits data. spi_receive_data() receives data. spi_peripheral_control() enables or disables SPI operation.

Application Example

The SPI driver was tested by transmitting the string 'HELLO STM32' using SPI1 configured as master. Communication can be verified using a logic analyzer. The SPI driver was tested by transmitting the string 'HELLO STM32' using SPI1 configured as master. Communication can be verified using a logic analyzer.

Advantages and Conclusion

The driver supports multiple configurations and demonstrates practical implementation of synchronous serial communication. The driver supports multiple configurations and demonstrates

practical implementation of synchronous serial communication.